Introduction to Operating Systems

Lecture Notes — Lecture 1

Functions, Evolution, Structure, Services, and System Calls

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Concepts That Click

1.1 Introduction

This chapter opens the course by asking a deceptively simple question: when you run a command such as `cp notes.txt backup.txt`, what actually happens between the keystroke and the disk? The obvious answer — "the computer does it" — hides an entire layer of software, the **operating system (OS)**, that sits between every program and every piece of hardware. Our goal in this chapter is to define that layer precisely, to see why it evolved the way it did, to understand how it is structured internally, and to learn how programs reach it through **system calls**.

A program is *logic*: a sequence of instructions that wants to read a file, send a packet, or use the CPU. The hardware is a set of *resources*: a processor, memory, disks, keyboards, network cards. The OS is the software that turns the raw resources into a safe, shared, usable machine. Throughout the chapter we follow one running thread, the command `cp notes.txt backup.txt`, tracing it from the shell all the way down to the disk controller and back. Every abstract idea we introduce gets anchored to that concrete trace.

1.2 What Is an Operating System?

1.2.1 Three Views of One Layer

There is no single-sentence definition of an operating system that captures everything it does, so it is more useful to keep three complementary views in mind. All three describe the same software from different angles.

Definition (The OS as resource manager). *A program that manages the computer's resources — the CPU, memory, storage, and I/O devices — deciding who gets what, and when.*

Definition (The OS as control program). *A program that controls the execution of user programs to prevent errors and improper use of the computer: the referee, not just the allocator.*

Definition (The OS as extended machine). *A program that hides the messy hardware behind clean abstractions — files, processes, sockets — so that programmers never have to touch a register or a disk sector directly.*

The three views emphasize different jobs. The resource-manager view matters when many programs compete for one machine: the OS multiplexes the CPU, allocates memory frames, and schedules disk requests. The control-program view matters for safety: the OS prevents one program from corrupting another or the kernel itself. The extended-machine view matters for the

programmer: it is what lets a single program run unchanged on very different hardware. Which view you lead with depends on what the machine is for, but a general-purpose OS must satisfy all three at once (Silberschatz et al., Ch. 1).

1.2.2 The Resource-Manager View in the Running Thread

Consider `cp notes.txt backup.txt`. To complete it, the CPU must execute the copy utility's instructions, memory must hold its buffer, the disk must store both files, and the I/O path must move the bytes. Each of these is a resource, and each can be wanted by several programs at the same time. The OS is what resolves the competition: it decides which process runs on the CPU now, which pages stay resident in memory, and in what order disk requests are served. For `cp`, the OS additionally *multiplexes* the resources — the same CPU runs `cp`, the shell, and every other process, one after another, fast enough to look simultaneous.

1.2.3 The Extended-Machine View: Why Abstraction Is the Point

A raw disk is an array of blocks addressed by a physical coordinate such as (cylinder, head, sector). No application programmer should have to know that geometry, and no program that did would survive a hardware change. The OS instead offers a **file**: a named stream of bytes that can be opened, read, written, and closed. The hardware details disappear. The same pattern repeats everywhere: a raw processor becomes a **process**, a network card becomes a **socket**, and physical memory frames become **virtual memory**. Abstraction is not decoration; it is what makes portability possible, because it *decouples* the program from the machine.

1.3 Where the OS Sits: The User/Kernel Boundary

The OS is not simply another library. It runs with privileges that user programs are denied, and the line between the two is enforced by hardware. We can picture the machine as four layers (Figure 1.1): user applications at the top, a system-call interface, the kernel, and the hardware at the bottom.

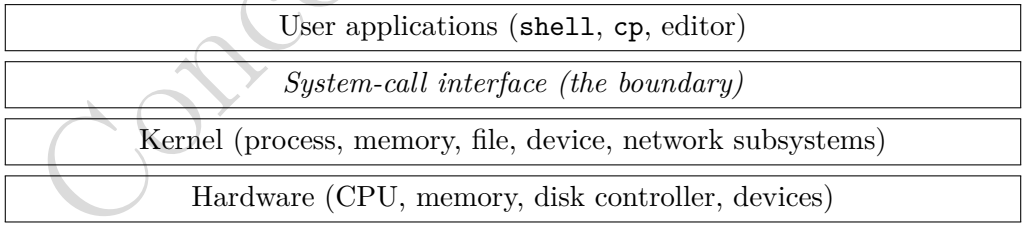


Figure 1.1: The user/kernel boundary. User programs reach the kernel only through the system-call interface.

Code runs in one of two modes. In **user mode**, privileged instructions are illegal; in **kernel mode**, they are allowed, and the OS itself runs here. The single legal doorway from user mode to kernel mode is the **system call**, a controlled trap that transfers control to the kernel at a known entry point. Everything that `cp` needs from the disk, the file system, or the scheduler must pass through that doorway.

Concepts That Click

1.4 Evolution of Operating Systems

The features of a modern OS did not appear at once. Each was a response to a real workload that the previous design handled badly, so the history doubles as a catalog of design pressures.

1.4.1 Serial Processing and Batch Systems

In the earliest **serial processing** systems (1940s–50s), the operator loaded one job, the program ran to completion, and then the next job began. There was essentially no OS; a human was the scheduler. The fatal flaw was utilization: the CPU sat idle while the operator mounted tapes and decks of cards. The **batch operating system** fixed this by grouping jobs of a similar type into a batch and running them under a **resident monitor** — a small program that transfers control from one job to the next automatically. A **job-control language (JCL)** told the monitor which programs to run and in what order. The lesson is durable: whenever the CPU waits on a slow human or device, an OS should take over the waiting.

1.4.2 Multiprogramming, Spooling, and Time-Sharing

Multiprogramming keeps several jobs in memory at once; when the running job waits for I/O, the OS switches to another, so the CPU stays busy. **Spooling** (Simultaneous Peripheral Operation On-Line) buffers input and output on disk so that slow devices run in parallel with the CPU — a printer no longer stops the computer. **Time-sharing** (multitasking) extends multiprogramming so that the CPU switches between users quickly enough that each has an *interactive* session, as if the machine were theirs alone. It is worth separating the two goals: multiprogramming maximizes *CPU utilization*, whereas time-sharing minimizes *response time*. On a busy machine the two can pull in opposite directions.

1.4.3 Interactive and Real-Time Systems

Interactive or time-sharing systems such as CTSS, Multics, and UNIX gave the user a shell and a prompt, and made the OS responsible for maintaining the illusion of private ownership. **Real-time systems** added a stronger requirement: correctness depends on *when* a result arrives, not only on its value. In a **hard real-time** system, a missed deadline is a failure (fly-by-wire controls, pacemakers, anti-lock brakes). In a **soft real-time** system, a missed deadline only degrades quality (video streaming, online games). A modern general-purpose OS such as Linux, Windows, or macOS tries to satisfy all three pressures — utilization, responsiveness, and predictability — at once.

1.5 System Protection

Protection is not a feature layered on top of the OS; it is the contract that makes everything else possible. If a user program could execute privileged instructions, one buggy program could overwrite the OS or another process. The defense rests on a small set of hardware mechanisms:

- **Dual-mode operation**. A hardware **mode bit** selects user mode (1) or kernel mode (0). Privileged instructions are legal only in kernel mode.
- **System call**. The only legal doorway from user mode to kernel mode — a controlled trap to the kernel.

Concepts That Click

- **Memory protection**. Base and limit registers, or page tables, ensure a process touches only its own memory.
- **Timer interrupt**. A hardware timer guarantees that the kernel regains control, so a runaway process cannot monopolize the CPU.

The timer deserves emphasis. Dual-mode operation alone would not be enough: a process could stay in user mode forever and never trap out. The timer is what makes preemption possible, because on every timer interrupt control returns to the kernel, which is free to schedule a different process.

1.6 Operating-System Structure

An OS is a very large, very long-lived program, and building one without a structure that survives decades is a losing battle. Five organizational styles dominate the landscape, and the trade-off across all of them is the same: simplicity and safety versus performance and flexibility.

1.6.1 Simple and Monolithic Structure

Early systems had no structure at all. MS-DOS, for example, was written so that its application programs could access the hardware directly, with no separation between the OS and user programs. The result was a system that was easy to build but fragile and insecure. Most UNIX kernels are *monolithic* in a milder sense: all the OS services live in one address space, so any part can call any other part with an ordinary procedure call. That is fast — there is no message-passing overhead — but it is also fragile, because there is no protection between subsystems and a bug in one can take down the whole kernel.

1.6.2 Layered Structure

A **layered** OS organizes the system as a hierarchy of levels, where level N may use only the facilities of level $N-1$. This makes the system easier to design and debug, because each layer can be verified against the layer below it before work begins on the layer above. The THE operating system used six layers, for example. The cost is twofold: layers are surprisingly hard to *define* cleanly, and each boundary adds call overhead.

1.6.3 Microkernel, Modular, and Virtual-Machine Structure

A **microkernel** pushes everything except a minimal core — inter-process communication, basic scheduling, and low-level memory management — out of the kernel and into user-space servers. Mach, QNX, and L4 are examples. The benefit is reliability, security, and portability: a crash in a user-space file server does not kill the kernel. The cost is performance, because what used to be a procedure call becomes a message passing through the microkernel.

A **modular** kernel takes a pragmatic middle path: the core of the OS is in the kernel, and additional services such as device drivers and file systems are *loadable modules* with defined interfaces. The core can call a module directly, while the module can be loaded and unloaded at runtime. This is essentially the structure of Linux today, and it captures much of the flexibility of a microkernel with most of the speed of a monolithic one.

A **virtual machine** takes a different tack altogether: a **hypervisor** presents one or more virtual copies of the hardware, and each guest runs its own operating system on top. VMware,

Concepts That Click

Xen, and KVM are examples. The isolation is near-perfect and consolidation is easy, but the design is complex, and I/O virtualization is expensive.

1.7 OS Services

Whatever its structure, an OS exposes a set of services. It is useful to split them by audience.

1.7.1 Services for the User

- **Program execution** — start, run, and stop programs.
- **I/O operations** and **file-system manipulation** — create, read, write, delete, and list files.
- **Communication** — move data between processes, either on one machine (shared memory) or across a network (message passing).
- **Error detection** — catch CPU, memory, I/O, and program errors and report them.

1.7.2 Services for System Efficiency

Beyond the user-facing services, the OS looks after its own house: **resource allocation** among concurrent users, **accounting** of who used how much, and **protection and security** to control which process may access what.

1.8 System Calls and System Programs

1.8.1 System Calls

A **system call** is a function that the OS provides to programs. When a program invokes one, a trap switches the processor from user mode to kernel mode, performs the privileged work, and returns. System calls come in families: **process control** (`fork`, `exec`, `wait`, `exit`), **file management** (`open`, `read`, `write`, `close`, `unlink`), **device management**, **information maintenance**, **communication**, and **protection**. Most application programming interfaces (the POSIX API, the Win32 API) *wrap* system calls, and a single library call may issue several of them. The API is what the programmer writes against; the system call is what the kernel actually executes.

1.8.2 The Running Thread: `cp notes.txt backup.txt`, Front to Back

We can now spell out the trace we have been promising.

1. `cp` calls `open("notes.txt", O_RDONLY)`. This is a **system call**: a mode switch into the kernel, which resolves the path through the directory structure and checks permission.
2. The kernel returns a **file descriptor** and has created a **file control block (FCB)** entry recording where the file's data lives.
3. `read` and `write` move bytes through a user-space buffer. Each call is a separate trap into the kernel.
4. For each block, the kernel issues a request to a **device driver**, which drives the disk **controller**.

Concepts That Click

5. `close` releases the descriptor. On success the shell prints nothing — success is silent.

The point of the trace is the ratio. Five user-visible system calls produce five mode switches, but behind them an enormous amount of OS machinery runs: path resolution, permission checks, buffer caching, scheduling, and device-driver logic — none of which the programmer wrote or even sees. In a real kernel, this same trace is the path from the system-call table through the virtual file system and down to the block-device driver; Bovet and Cesati follow that path line by line for Linux (Ch. 10, system calls; Ch. 12, the virtual filesystem).

1.8.3 Passing Parameters

Because a system call crosses a mode boundary, the kernel and the program must agree on how arguments are passed. There are three standard mechanisms: through **registers** (simplest, but limited by the number of registers and the size of the arguments); through a **block or table** whose address is passed in a register (used by Linux and Solaris); or through the **stack**, where the program pushes the parameters and the OS pops them.

1.8.4 System Programs

Finally, users rarely invoke system calls directly. They use **system programs**: user-level tools built on top of the system calls, such as shells, compilers, file managers, editors, and the utilities in `/bin`. These are not part of the kernel, but they are part of the operating system as a user experiences it. This is precisely why the shell is not kernel code: it is a program that issues system calls, and yet it is the visible face of the whole system.

1.9 Summary

An operating system is best understood as three things at once: the resource manager that multiplexes the machine, the control program that protects it, and the extended machine that abstracts it. Its history is a response to three pressures — utilization (batch and multiprogramming), responsiveness (time-sharing), and predictability (real time). The boundary between user and kernel mode, backed by the timer and by memory protection, is what makes the whole arrangement safe. Internally the OS is organized along a spectrum from monolithic through layered to microkernel, modular, and virtual-machine designs, trading simplicity against performance. And every program reaches the OS in exactly one way: through system calls, wrapped in the convenience of system programs.

Self-Check

1. Give the three definitions of an operating system and one concrete situation in which each is the most useful view.

2. A process enters an infinite loop with no I/O. Why does the machine not hang forever? Name the mechanism.

3. What is the difference between multiprogramming and time-sharing in terms of the objective each optimizes?

Concepts That Click

4. Why is a microkernel considered more reliable than a monolithic kernel, and what price does it pay?

5. In the `cp` trace, at which step does the first user/kernel mode switch occur, and what triggers it?

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 8th ed., John Wiley and Sons — Ch. 1 (pp. 3–47), Ch. 2 (pp. 49–89). *(Page-cited; index built 2026-10-14.)*

[2] W. Stallings, *Operating Systems: Internals and Design Principles*, 6th ed., Prentice Hall — Ch. 1–3. *(Page-cited; index built 2026-10-14.)*

[3] M. J. Bach, *The Design of the UNIX Operating System*, Prentice Hall of India, 1986.

[4] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed., O'Reilly Media, 2006 — Ch. 1 (basic concepts, files/inodes), Ch. 10 (system calls). *(Page-cited; index built 2026-10-14.)*

Concepts That Click